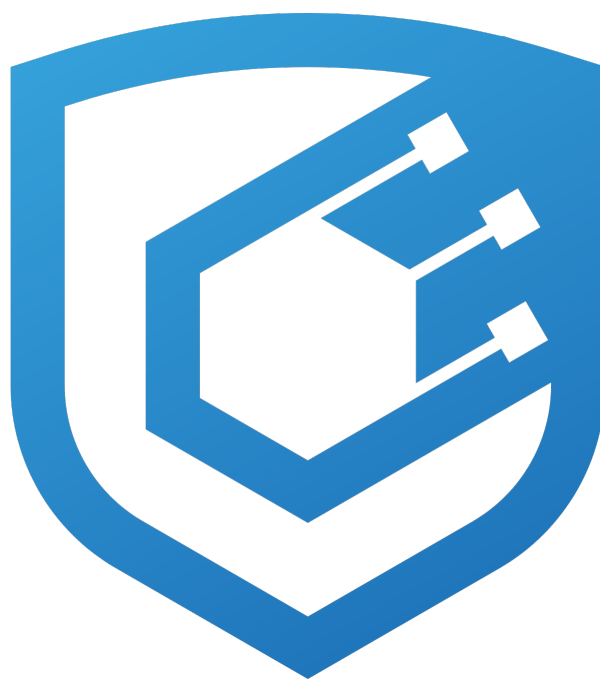




Predict.fun Chainlink CRE Integration Audit Report

Prepared by [Cyfrin](#)

Version 2.1

Lead Auditors

[Jorge](#)

[Raiders](#)

April 25, 2026

Contents

1	About Cyfrin	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
5	Audit Scope	3
6	Executive Summary	3
6.1	Trust Assumptions and Centralization Risks	4
6.1.1	Role Architecture	4
6.1.2	Administrative Capabilities	4
6.1.3	Centralization Disclosures	4
6.1.4	Deployment Notes	5
6.1.5	External Trust Dependencies	5
7	Findings	8
7.1	Medium Risk	8
7.1.1	prices[feedId][timestamp] key omits interval — admin-supplied startPrice silently dropped on cross-interval slot collision	8
7.1.2	CRE writeCloseRounds ignores txStatus, receiverContractExecutionStatus, and errorMessage	9
7.1.3	Single bad report halts whole batch - no try/catch on cron tick, aggregate3 non-resilient	11
7.1.4	Stuck round when isEnabled == false and last pre-created round closes	13
7.1.5	enableRoundConfig uses <= instead of <, forcing an unnecessary one-interval gap when re-enabling a round series	15
7.2	Low Risk	16
7.2.1	setExpectedAuthor(address(0)) with non-empty workflow name causes onReport DoS	16
7.2.2	ChainlinkUpDownAdapter::emergencyCloseRounds accepts empty price array and negative prices	16
7.2.3	Missing runtime input validation in admin setters and extension paths	17
7.2.4	Missing CRE-side runtime validation of config values before use	17
7.2.5	Inconsistent expiry comparison between CRE pre-flight and onchain validation	19
7.2.6	enableRoundConfig skips _validateInterval check allowing re-activation for invalidated intervals	19
7.3	Informational	21
7.3.1	chainlinkReports[i].interval is caller-controlled, not in the DON-signed payload	21
7.3.2	ChainlinkAdapter::_constructPayouts silent [0,0] fallback on unexpected outcome	21
7.3.3	Missing whenNotPaused on onReport pre-validation and no reentrancy guards on CTF calls	21
7.3.4	Event emission gaps and telemetry asymmetries	21
7.3.5	Do not initialize local variables to their default values	22
7.3.6	Deployer EOA retains DEFAULT_ADMIN_ROLE after deployment	22
7.3.7	Pre-deploy placeholders and deploy-script gaps: unset roles, placeholder addresses and stale timestamps	23
7.3.8	Documentation issues: NatSpec/code mismatches, missing @inheritdoc and missing clarifying comments	24
7.3.9	Code style and naming conventions: inconsistent loops, storage prefixes, magic numbers and conflicting names	25
7.3.10	Missing events: constructor init event absent and admin-mutating events lack msg.sender	26
7.3.11	Missing deploy-time input validation in constructor and initializer	26
7.3.12	emergencyCloseRounds propagates admin-supplied endPrice as next round's startPrice allowing outcome predetermination	27
7.4	Gas Optimization	29

7.4.1	Cache storage array length	29
7.4.2	Emit before state change or use input value to avoid storage re-read	29
7.4.3	Cache roundConfigIndices[feedId][interval] inside _createRound loops	30
7.4.4	ChainlinkUpDownAdapter::enableRoundConfig reads roundConfigIndices[feedId][interval] twice	30
7.4.5	Fail-fast: move cheap input checks before storage reads	31
7.4.6	Explicit payouts[0] = 0 and payouts[1] = 0 writes a zero value	31
7.4.7	ChainlinkReceiverBase::onReport storage re-reads of optional permission fields	31

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Predict.fun is a prediction market platform previously [audited](#) by Cyfrin.

5 Audit Scope

The focus of this audit was [PR71](#) which added Chainlink CRE integration; the audit scope was limited to:

```
// solidity
contracts/Adapters/Chainlink/ChainlinkAdapter.sol
contracts/Adapters/Chainlink/ChainlinkReceiverBase.sol
contracts/Adapters/Chainlink/ChainlinkUpDownAdapter.sol

// chainlink CRE
cre/data-stream-resolution/close-rounds/fetchDataStreamsReportsBulk.ts
cre/data-stream-resolution/close-rounds/main.ts
cre/data-stream-resolution/close-rounds/onCronTrigger.ts
cre/data-stream-resolution/close-rounds/readRoundConfigs.ts
cre/data-stream-resolution/close-rounds/writeCloseRounds.ts
```

6 Executive Summary

Over the course of 4 days, the Cyfrin team conducted an audit on the [Chainlink CRE Integration](#) code provided by [Predict.fun](#). The findings consist of 5 Medium & 6 Low severity issues with the remainder being informational and gas optimizations.

6.1 Trust Assumptions and Centralization Risks

The predict.fun Chainlink CRE adapter is built on OpenZeppelin `AccessControl` with a deliberately layered role model. The access control architecture is well-considered: roles are separated by function, capabilities are clearly scoped, and the deployment configuration is consistent with the threat model described in the protocol's technical documentation. The disclosures below are standard transparency artefacts for a role-administered protocol they describe capabilities inherent to the chosen architecture, not deficiencies in its implementation.

6.1.1 Role Architecture

The adapter defines five roles. The `DEFAULT_ADMIN_ROLE` is held by the multi-sig at `0x7625B1c374b0D1546d87c0ab66229461007f1e33` on BSC mainnet, which retains the ability to self-grant any subordinate role via OpenZeppelin `AccessControl` defaults. The `INITIALIZER_ROLE` is also held by the multi-sig and permits creation of new `(feedId, interval)` round series. The `EXTENDER_ROLE`, which permits pre-creation of additional rounds on an active series, is currently assigned to `address(0)` and is therefore uncallable on mainnet until the admin grants it to a real operator. Neither `PAUSER_ROLE` nor `EMERGENCY_CLOSE_ROUND_ROLE` is assigned at deployment; exercising the emergency path requires the admin to first self-grant these roles, which is a deliberate design choice that adds one step of ceremony before the emergency path can be used.

6.1.2 Administrative Capabilities

The following capabilities are available to the `DEFAULT_ADMIN_ROLE` holder. Each is an intentional design feature; they are listed for completeness.

Settlement control. The admin can pause the adapter, blocking CRE-driven settlement. While paused, `emergencyCloseRounds` permits settlement of open rounds using admin-supplied `int192` end prices. A `PriceMismatch` guard prevents overwriting a round slot that already holds a Chainlink-derived value; for rounds not yet settled by the CRE, no oracle cross-check applies. This path is designed for recovery when the CRE is unavailable.

Market configuration. The admin can create or re-enable round series, choosing the `startTimestamp` and initial `startPrice`. The start price is not oracle-verified at creation time; it becomes the baseline against which the first CRE-delivered end price is compared. `enableRoundConfig` now validates the interval via `_validateInterval`, consistent with `initializeRoundSeries` and `extend`. The timestamp boundary check uses strict `<`, allowing restart at the exact natural series boundary without a forced one-interval gap.

CRE routing. The admin can rotate the Keystone forwarder address and manage the workflow identity whitelist. The updated `setIsWorkflowAuthorAndNameValid` atomically sets the `(author, workflowName)` pair and enforces that neither component is null, removing a prior asymmetric-state condition.

6.1.3 Centralization Disclosures

The capabilities above give the `DEFAULT_ADMIN_ROLE` holder material influence over round outcomes under adversarial assumptions. These are architecture-level trust assumptions, not code vulnerabilities. They are disclosed in full below.

The admin can pause the adapter and settle any open round in any active series via `emergencyCloseRounds` with arbitrary prices, directly determining outcomes for rounds not yet resolved by the CRE. The `PriceMismatch` guard applies only to slots already written by the CRE.

The admin's chosen `startPrice` for a new or re-enabled series is unverified. A value that diverges from the prevailing market price will bias the first round's outcome toward whichever direction the deviation favours.

An admin with knowledge of a pending CRE report could, in principle, submit an `emergencyCloseRounds` call with a preferred outcome before the CRE transaction lands. The CRE report would then revert on the already-advanced round state.

The practical risk is governed by the operational security of the admin multi-sig and the protocol team's change-control processes. Users should understand that the emergency close mechanism is a documented recovery capability, not an oracle-verified settlement path.

6.1.4 Deployment Notes

The constructor grants `DEFAULT_ADMIN_ROLE` to `msg.sender`; the deployment script grants the same role to the multi-sig but does not revoke the deployer's grant. Post-deployment, two principals can administer the contract: the multi-sig (intended) and the deployer EOA (unintended). The protocol team has confirmed that the deployer role is revoked via a post-deployment multi-sig transaction. On-chain verification of this revocation before market opening is recommended.

The `EXTENDER_ROLE` is assigned to `address(0)` at deployment. The `extend` function is currently uncallable on mainnet. When an operator address is designated, the admin can grant the role via standard `AccessControl`.

6.1.5 External Trust Dependencies

Settlement relies on three Chainlink-operated systems outside Predict.fun's direct control. Extended unavailability of any of them shifts settlement responsibility to the admin-controlled emergency path.

Chainlink CRE workflow is the sole healthy-path settlement source. The workflow now includes retirement detection, configurable batch sizing, and runtime configuration validation. Settlement requires byte-identical Data Streams responses across all DON nodes; any variance causes the consensus step to fail and the affected rounds to remain open until the next cron tick or until emergency settlement is invoked.

Chainlink Keystone Forwarder (`0x76c9cf548b4179F8901cda1f8623568b58215E62`, BSC mainnet) is operated by Chainlink. Signer-set rotations may transiently invalidate in-flight reports; brief settlement delays during upstream maintenance are expected and tolerable.

Chainlink Verifier Proxy (`0xF276a4BC8Da323EA3E8c3c195a4E2E7615a898d1`, BSC mainnet), the adapter calls `verifyBulk` with an empty `parameterPayload`, which currently bypasses the fee manager because none is configured on BSC. The `VERIFIER_PROXY` address is immutable; if Chainlink introduces fee manager requirements on BSC in the future, `verifyBulk` calls would revert and a redeployment would be required to resolve it. This dependency is outside Predict.fun's control and is flagged as an operational watch item.

Chainlink Data Streams V3, report `expiresAt` values carry approximately a 30-day TTL for crypto spot feeds, making the window between CRE execution and on-chain inclusion negligible in normal operation. The adapter enforces `block.timestamp < expiresAt` at settlement time.

Summary

Project Name	Chainlink CRE Integration
Repository	prediction-market
Commit	0a580a178f1b...
Fix Commit	e21160d6dc3a...
Audit Timeline	Apr 20th - Apr 23rd, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	5
Low Risk	6
Informational	12
Gas Optimizations	7
Total Issues	30

Summary of Findings

[M-1] prices[feedId][timestamp] key omits interval — admin-supplied startPrice silently dropped on cross-interval slot collision	Resolved
[M-2] CRE writeCloseRounds ignores txStatus, receiverContractExecutionStatus, and errorMessage	Resolved
[M-3] Single bad report halts whole batch - no try/catch on cron tick, aggregate3 non-resilient	Resolved
[M-4] Stuck round when isEnabled == false and last pre-created round closes	Resolved
[M-5] enableRoundConfig uses <= instead of <, forcing an unnecessary one-interval gap when re-enabling a round series	Resolved
[L-1] setExpectedAuthor(address(0)) with non-empty workflow name causes onReport DoS	Resolved
[L-2] ChainlinkUpDownAdapter::emergencyCloseRounds accepts empty price array and negative prices	Resolved
[L-3] Missing runtime input validation in admin setters and extension paths	Resolved
[L-4] Missing CRE-side runtime validation of config values before use	Resolved
[L-5] Inconsistent expiry comparison between CRE pre-flight and onchain validation	Resolved
[L-6] enableRoundConfig skips _validateInterval check allowing re-activation for invalidated intervals	Resolved
[I-01] chainlinkReports[i].interval is caller-controlled, not in the DON-signed payload	Acknowledged
[I-02] ChainlinkAdapter::_constructPayouts silent [0,0] fallback on unexpected outcome	Resolved
[I-03] Missing whenNotPaused on onReport pre-validation and no reentrancy guards on CTF calls	Acknowledged
[I-04] Event emission gaps and telemetry asymmetries	Acknowledged
[I-05] Do not initialize local variables to their default values	Acknowledged
[I-06] Deployer EOA retains DEFAULT_ADMIN_ROLE after deployment	Acknowledged

[I-07] Pre-deploy placeholders and deploy-script gaps: unset roles, placeholder addresses and stale timestamps	Resolved
[I-08] Documentation issues: NatSpec/code mismatches, missing @inheritdoc and missing clarifying comments	Resolved
[I-09] Code style and naming conventions: inconsistent loops, storage prefixes, magic numbers and conflicting names	Resolved
[I-10] Missing events: constructor init event absent and admin-mutating events lack msg.sender	Resolved
[I-11] Missing deploy-time input validation in constructor and initializer	Acknowledged
[I-12] emergencyCloseRounds propagates admin-supplied endPrice as next round's startPrice allowing outcome predetermination	Acknowledged
[G-1] Cache storage array length	Resolved
[G-2] Emit before state change or use input value to avoid storage re-read	Partially Resolved
[G-3] Cache roundConfigIndices[feedId][interval] inside _createRound loops	Acknowledged
[G-4] ChainlinkUpDownAdapter::enableRoundConfig reads roundConfigIndices[feedId][interval] twice	Acknowledged
[G-5] Fail-fast: move cheap input checks before storage reads	Resolved
[G-6] Explicit payouts[0] = 0 and payouts[1] = 0 writes a zero value	Acknowledged
[G-7] ChainlinkReceiverBase::onReport storage re-reads of optional permission fields	Resolved

7 Findings

7.1 Medium Risk

7.1.1 `prices[feedId][timestamp]` key omits interval — admin-supplied `startPrice` silently dropped on cross-interval slot collision

Description: `ChainlinkUpDownAdapter.sol:37` declares `mapping(bytes32 feedId => mapping(uint32 timestamp => int192 price)) prices` — keyed only on `(feedId, timestamp)`, omitting both interval and `roundConfigIndices`. `_setNextRoundStartPriceIfPresent` at `ChainlinkUpDownAdapter.sol:317-323` only writes when the slot is zero; otherwise it silently keeps the existing value and emits `RoundStartPriceSet` with that stale number.

When both 5m (300) and 15m (900) are active on the same feed (the stated protocol config), every 15m boundary `T` is also a 5m boundary. In normal CRE-only operation this is harmless because both reports at `observationTimestamp = T` carry the identical `report.price` from the same Chainlink Data Stream (see the `_closeRound` call site at `ChainlinkUpDownAdapter.sol:245`, which passes `report.price` verbatim as the `endPrice`) — both writes store the same value.

The slot collision **does** corrupt state, however, whenever one of the two writes is an admin-supplied `startPrice` rather than a Chainlink-derived `report.price`:

1. **initialize-time collision** — admin calls `initialize(feedId, interval=15m, startTimestamp=T, startPrice=admin_price_from_Binance, ...)` on a feed whose 5m config was initialized earlier and whose CRE flow has already written `prices[feedId][T]` as a 5m round's `endPrice` (`T` being a past timestamp that is simultaneously a 5m and 15m boundary). `_initialize` calls `_setNextRoundStartPriceIfPresent` at `ChainlinkUpDownAdapter.sol:400`, which finds the slot non-zero, silently keeps the Chainlink-derived value, and discards `admin_price_from_Binance`. The 15m config's first round then resolves against the Chainlink price instead of the admin-chosen price referenced in the inline comment at line 376 ("This is to make sure the `startPrice` is something that actually comes from Binance API.").
2. **enableRoundConfig-time collision** — same mechanic via `enableRoundConfig` → `_initialize` → `_setNextRoundStartPriceIfPresent`. The admin-chosen `startPrice` is silently dropped if another interval's CRE flow has already written the slot.
3. **Reverse collision** — after admin writes `prices[feedId][T] = admin_price` via `initialize/enableRoundConfig` for one interval, a subsequent CRE close of the other interval's round ending at the same `T` silently no-ops (slot already non-zero). That other interval's next round, which starts at `T`, then reads `admin_price` as its `startPrice` instead of the Chainlink-derived `endPrice`. Any divergence between the admin's source (Binance) and Chainlink at `T` — typically small but non-zero — maps directly onto the Up/Down outcome on tight rounds.

In all three variants, `RoundStartPriceSet` emits the *stored* value rather than the *supplied* one, so the discrepancy is invisible off-chain: operators see a benign-looking event carrying the previously-stored price and have no signal that their intended value was discarded.

Impact: Admin-supplied `startPrice` (manually sourced per the line 376 comment) is silently replaced by a Chainlink-derived value — or vice versa — at any timestamp where two intervals for the same feed coincide. Rounds that resolve against the wrong reference price Up/Down-flip on tight markets, mispaying 100% of CTF liquidity on that round. The stated invariant "start price is always the previous round's end price" holds in the CRE-only steady state but is violated at every config boundary where an admin price is supplied.

Proof of Concept: Preconditions

- Both `updateIsIntervalValid(300, true)` and `updateIsIntervalValid(900, true)` executed.
- Mapping `prices[feedId][timestamp]` at `ChainlinkUpDownAdapter.sol:37` is keyed only on `(feedId, timestamp)`.

Attack sequence — reverse collision (admin writes first, CRE close silently no-ops)

1. Admin calls `initialize(BTC/USD, 900, T, admin_price, ...)` where `T` is a past 15m boundary (also a 5m boundary) and the 5m interval has not yet written that slot. `_initialize` writes `prices[BTC/USD][T] =`

admin_price.

2. 5m CRE closes its round ending at T with `report.price = P_chainlink ( admin_price` because the two come from different sources). `_setNextRoundStartPriceIfPresent` finds the slot populated with `admin_price`, silently declines to overwrite, and emits `RoundStartPriceSet(BTC/USD, 300, T, admin_price)` — making it look as though `admin_price` were the 5m close.
3. The 5m round that starts at T closes at T + 300 with endPrice `P_end`. It reads `startPrice = admin_price` and resolves Up/Down by comparing `P_end` against the admin's Binance price rather than the 5m round's true previous endPrice.

Attack sequence — initialize-time collision (CRE wrote first, admin's value silently dropped)

1. 5m CRE has already closed a round ending at past 15m boundary T, populating `prices[BTC/USD][T] = P_chainlink`.
2. Admin calls `initialize(BTC/USD, 900, T, admin_price, ...)`. `_initialize's _setNextRoundStartPriceIfPresent` finds the slot non-zero, silently keeps `P_chainlink`, and emits `RoundStartPriceSet(..., P_chainlink)`.
3. The 15m config's first round closes against `P_chainlink`, not `admin_price`. The admin's chosen reference is silently overridden and off-chain observers cannot distinguish this from the intended path.

Expected outcome

- Either direction of collision leaves the prices slot containing whichever value was written first, regardless of which was intended.
- `RoundStartPriceSet` reports the stored value rather than the supplied value, so off-chain monitoring cannot detect the silent discard.
- Any round that spans the affected boundary uses the wrong reference, flipping outcomes on tight rounds.

Recommended Mitigation: Change the mapping key to include both `interval` and the config index, mirroring `roundKey`:

```
// Option A: key by the full roundKey
mapping(bytes32 roundKey => int192 price) public prices;
// writes/reads become:
prices[roundKey(feedId, interval, roundConfigIndices[feedId][interval], startTimestamp)] = endPrice;

// Option B: nest interval under feedId and timestamp
mapping(bytes32 feedId => mapping(uint32 interval => mapping(uint32 timestamp => int192))) public
prices;
```

Additionally fix the misleading event — emit the supplied `startPrice`, not the stored value:

```
emit ChainlinkUpDownAdapter__RoundStartPriceSet(feedId, interval, startTimestamp, startPrice);
```

or add an explicit "already-present, not overwritten" event distinct from "freshly set".

Predict.fun: Fixed in commit [99b763](#).

Cyfrin: Verified.

7.1.2 CRE `writeCloseRounds` ignores `txStatus`, `receiverContractExecutionStatus`, and `errorMessage`

Description: `cre/data-stream-resolution/close-rounds/writeCloseRounds.ts` consumes only `txHash` from the `WriteReportReply` returned by `evmClient.writeReport`. On-chain reverts populate `txStatus`, `receiverContractExecutionStatus`, and `errorMessage`, but the CRE treats every reply as success. Any persistent revert (expired reports, missing `PAUSER_ROLE` / `EMERGENCY_CLOSE_ROUND_ROLE`, config drift, questionID collision, strict-equality failures) is silently invisible to operators.

Impact: Each failed tick consumes gas up to the configured `gasLimit` of 2M. At the production cron schedule of every 10 seconds (8,640 ticks/day per `* / 10 * * * * *`) & BNB's current median gas price of 0.1 Gwei - a

persistent failure burns up to 1.728 BNB/day per feed and the dollar cost scales with prevailing BNB market price and number of active feeds.

Proof of Concept: Preconditions

- Adapter deployed, CRE live, forwarder whitelisted.
- Some adapter-side precondition broken in production (role missing, workflow id mismatch, expired-report drift, questionID collision, strict-equality timestamp mismatch).
- writeCloseRounds.ts reads only { txHash }, ignoring other WriteReportReply fields.
- gasLimit = 2_000_000.

Failure sequence

1. Cron tick fetches valid DS reports; workflow packages ChainlinkReport[] with correct intervals.
2. evmClient.writeReport **broadcasts** a Keystone forwarder tx with the batch. Reply shape:

```
interface WriteReportReply {
  txHash: string;
  txStatus: "SUCCESS" | "REVERTED" | "FAILED";
  receiverContractExecutionStatus: "EXECUTED" | "REVERTED" | "SKIPPED";
  errorMessage: string;
}
```

3. onReport **reverts on-chain** (e.g. ChainlinkAdapter__ReportExpired because block.timestamp > report.expiresAt by mining time). Keystone forwarder tx succeeds at top level but receiverContractExecutionStatus = "REVERTED" and errorMessage = "ChainlinkAdapter__ReportExpired".
4. writeCloseRounds.ts **reads only** txHash:

```
const { txHash } = await evmClient.writeReport(runtime, payload);
log.info(`Submitted close-rounds tx ${txHash}`);
return { ok: true, txHash };
```

No check of txStatus / receiverContractExecutionStatus / errorMessage. Caller sees success.

5. **Next tick repeats identical failure** — currentRoundStartTimestamp never advanced, so same (feedId, interval, roundEnd) tuple. Systemic failures (missing role, wrong workflowId) persist indefinitely.
6. **Dashboard shows green.** Grafana, PagerDuty, log aggregators all see { ok: true }. Only manual on-chain inspection reveals the outage.

Expected outcome

- Indefinite silent settlement halt.
- Burn of up to 1.728 BNB/day per feed (2,000,000 gasLimit × 0.1 Gwei × 8,640 ticks/day); dollar cost scales with prevailing BNB price and number of active feeds.
- Operator MTTD dominated by manual discovery (days).

Recommended Mitigation: Check all three reply fields and surface failures:

```
const reply = await evmClient.writeReport(runtime, payload);
if (reply.txStatus !== "SUCCESS" || reply.receiverContractExecutionStatus !== "EXECUTED") {
  runtime.log.error(
    `writeReport failed: tx=${reply.txHash} txStatus=${reply.txStatus} ` +
    `exec=${reply.receiverContractExecutionStatus} err=${reply.errorMessage}`
  );
  return { ok: false, txHash: reply.txHash, error: reply.errorMessage };
}
return { ok: true, txHash: reply.txHash };
```

Emit a metric / alert on each { ok: false } and integrate into the operator dashboard so silent reverts trip a paging threshold.

Predict.fun: Fixed in commits [fc07087](#) & [bfa24e2](#).

Cyfrin: Verified.

7.1.3 Single bad report halts whole batch - no try/catch on cron tick, aggregate3 non-resilient

Description: Three stacked atomicity failures cascade a single transient error to a full-batch outage:

- **Solidity side:** ChainlinkUpDownAdapter::_processReport at ChainlinkUpDownAdapter.sol:241-... decodes the ChainlinkReport[] atomically. VERIFIER_PROXY.verifyBulk is single-call; _validateReportTimestamp(report) is invoked per entry inside a single Solidity loop; any revert halts the whole loop and reverts the whole transaction. One expired / one NotCurrentRound / one failing verifyBulk entry for a single (feed, interval) pair means every other pair also fails.
- **CRE** onCronTrigger.ts: lacks try/catch around readRoundConfigs and downstream — any feed's RPC hiccup or single decode error throws the whole tick.
- **CRE** readRoundConfigs.ts: uses aggregate3({ allowFailure: false }) — any sub-call failure reverts the entire eth_call.
- **CRE** readRoundConfigs.ts: per-report HTTP response validation uses bare throw — if any report in the bulk response has expiresAt < nowSeconds (lines 94-96) or observationsTimestamp !== endTimestamp (lines 98-101), the error propagates uncaught through onCronTrigger and aborts the entire tick. These are timing-sensitive conditions that can be triggered by normal Data Streams API latency, making them the most likely of all three CRE failure modes to fire in production

Impact: One feed's transient failure cascades to halt all feeds every tick. No funds are permanently at risk and no state will be corrupted so the actual harm is: delayed settlement (one missed tick per failure event) and total silence about which entry caused the failure. This linearly worsens as protocol scales (client concern 3).

Proof of Concept: Scenario A — one expired report halts the Solidity batch

1. **Cron tick at $t = T$.** CRE has fetched 4 ReportV3s for (BTC,300), (BTC,900), (ETH,300), (ETH,900). The ETH 5m report was fetched near its expiresAt due to DS latency; by the time the tx mines, block.timestamp > reportETH.expiresAt.

2. **Batch submitted to onReport:**

```
forwarder -> adapter.onReport(metadata, encode(ChainlinkReport[] = [
  { interval: 300, rawReport: btcReport },
  { interval: 900, rawReport: btcReport15m },
  { interval: 300, rawReport: ethReport },    // expired
  { interval: 900, rawReport: ethReport15m }
]));
```

3. **_processReport verifies the batch** (verifyBulk returns 4 decoded entries — signature verification orthogonal to expiry).
4. **Per-entry loop hits the expired report** at $i = 2$: _validateReportTimestamp reverts with ChainlinkAdapter__ReportExpired.
5. **All 4 rounds fail to close.** Tx reverts; currentRoundStartTimestamp does not advance for ANY pair. Legitimate BTC reports thrown out.
6. **Next tick repeats** if the latency is systemic.

Scenario B — readRoundConfigs RPC failure halts the tick before any work

1. **Cron tick at $t = T$.** onCronTrigger starts.
2. **Multicall3 batch forwards 4 calls.**

3. **One sub-call returns malformed data** (RPC hiccup, proxy corruption, truncated bytes, or a rotated round-Key). With `allowFailure: false`, `aggregate3` reverts the whole `eth_call`. Alternatively, subcall succeeds but returns malformed bytes — `viem`'s `decodeFunctionResult` throws.
4. **Throw propagates to `onCronTrigger`** — no try/catch -> CRE harness discards the tick. ZERO reports submitted, even for the 3 pairs whose state was fine.

Scenario C — HTTP report validation failure halts Cron tick at $t = T$. `onCronTrigger` has successfully read round configs (Step 1) and identified N ready-to-close rounds (Step 2). It proceeds to Step 4 and calls `fetchDataStreamsReportsBulk` for a given `endTimeStamp` bucket.

The Chainlink Data Streams HTTP API returns the bulk response, but one of the N reports in the response fails CRE-side validation inside `fetchDataStreamsReportsBulk`

```
if (decodedData.expiresAt < nowSeconds) {
    throw new Error(`Report expired for ${report.feedID}: expiresAt=${decodedData.expiresAt},
    ↪ now=${nowSeconds}`);
}

if (report.observationsTimestamp !== endTimeStamp) {
    throw new Error(
        `Report observationsTimestamp mismatch for ${report.feedID}: expected=${endTimeStamp},
        ↪ got=${report.observationsTimestamp}`,
    );
}
```

Expected outcome

- Linear amplification: one failing feed halts N feeds. Blast radius grows with feed count.
- Hundreds of YES/NO positions unresolved past intended settlement times per incident.

Recommended Mitigation:

1. **Solidity — handle each report atomically with try/catch-equivalent fallback:** refactor the per-entry loop to continue on per-entry validation failure, emitting an event per skipped report, so one stale entry does not block the rest:

```
for (uint256 i; i < chainlinkReports.length; ) {
    ReportV3 memory r = abi.decode(verified[i], (ReportV3));
    if (block.timestamp >= r.expiresAt || block.timestamp < r.observationsTimestamp) {
        emit ChainlinkUpDownAdapter__ReportSkipped(r.feedId, chainlinkReports[i].interval,
        ↪ r.observationsTimestamp);
        unchecked { ++i; } continue;
    }
    _closeRound(r.feedId, chainlinkReports[i].interval, r.observationsTimestamp -
    ↪ chainlinkReports[i].interval, r.price);
    unchecked { ++i; }
}
```

(Note: `verifyBulk` atomicity remains — signature-level failures still halt, which is acceptable since that indicates compromise. Only timestamp/round-state failures should be per-entry tolerant.)

2. **CRE — wrap `onCronTrigger` in try/catch** with per-feed isolation so one feed's RPC failure does not halt others:

```
for (const [feedId, interval] of pairs) {
    try {
        await processPair(feedId, interval);
    } catch (err) {
        runtime.log.error(`pair ${feedId}/${interval} failed: ${err.message}`);
    }
}
```

3. `readRoundConfigs` — **use** `allowFailure: true` and skip / alert on per-call failures rather than aborting the whole batch.
4. **`fetchDataStreamsReportsBulk`** — replace per-report throw on validation failure with a skip-and-log pattern so one stale or mismatched report does not discard the entire bulk result:

```
if (decodedData.expiresAt < nowSeconds) {
  runtime.log(`Skipping expired report for ${report.feedID}: expiresAt=${decodedData.expiresAt},
    ↳ now=${nowSeconds}`);
  continue;
}
if (report.observationsTimestamp !== endTimestamp) {
  runtime.log(`Skipping timestamp mismatch for ${report.feedID}: expected=${endTimestamp},
    ↳ got=${report.observationsTimestamp}`);
  continue;
}
```

Predict.fun: Fixed in commits [29a4d25](#), [894768](#).

Cyfrin: Verified.

7.1.4 Stuck round when `isEnabled == false` and last pre-created round closes

Description: When `disableRoundConfig` is called, it sets `roundConfig.isEnabled = false` but leaves `currentRoundStartTimestamp` and `latestRoundStartTimestamp` unchanged. The Chainlink CRE offchain workflow (`readRoundConfigs.ts`) only reads `currentRoundStartTimestamp` to decide which rounds to close — it never reads or checks `isEnabled`. As a result, the CRE keeps submitting close-round reports for a disabled config exactly as if it were active.

The critical failure occurs when the CRE closes the last pre-created round of a disabled config (the one at `latestRoundStartTimestamp`). At that point, inside `_closeRound`:

```
function _closeRound(bytes32 feedId, uint32 interval, uint32 startTimestamp, int192 endPrice) private {
  ...

  uint32 nextRoundStartTimestamp = startTimestamp + interval;
  bool present = _setNextRoundStartPriceIfPresent(feedId, nextRoundStartTimestamp, interval,
    ↳ endPrice);
  if (present) {
    roundConfig.currentRoundStartTimestamp = nextRoundStartTimestamp;
  }
  if (roundConfig.isEnabled) {
    uint32 newLatestTimestamp = roundConfig.latestRoundStartTimestamp + interval;
    roundConfig.latestRoundStartTimestamp = newLatestTimestamp;
    _createRound(feedId, newLatestTimestamp, interval);
  }
}
```

Because `isEnabled = false`, no next round was ever pre-created, so `_setNextRoundStartPriceIfPresent` returns false, and `currentRoundStartTimestamp` is never advanced. It permanently points to the round that was just resolved.

The CRE offchain workflow reads the on-chain state in `readRoundConfigs.ts` through two Multicall3 batches. Batch 1 fetches `roundConfigIndices` (unchanged by disable), batch 2 fetches `roundConfigs` by key and decodes the result — but only extracts `currentRoundStartTimestamp`, the first field of the struct. `isEnabled` is never read:

```
// readRoundConfigs.ts - Batch 1: read roundConfigIndices (unchanged after disable)
indexCalls.push({
  target: adapterAddress,
  allowFailure: false,
  callData: encodeFunctionData({
    abi: ChainlinkUpDownAdapter,
```

```

    functionName: "roundConfigIndices",
    args: [feedId, interval],
  }},
});

// ...

// Batch 2: read roundConfigs by key
configCalls.push({
  target: adapterAddress,
  allowFailure: false,
  callData: encodeFunctionData({
    abi: ChainlinkUpDownAdapter,
    functionName: "roundConfigs",
    args: [key],
  }),
});

// ...

// Only currentRoundStartTimestamp is decoded - isEnabled is never read
const [currentRoundStartTimestamp] = decoded;
return { feedId, interval, currentRoundStartTimestamp: BigInt(currentRoundStartTimestamp) };

```

Back in `onCronTrigger.ts`, the only two guards applied to the returned timestamps are a zero-check and a time check. A disabled config satisfies both, so it is treated identically to an active one:

```

// onCronTrigger.ts - Step 2: filter ready rounds
for (const { feedId, interval, currentRoundStartTimestamp } of roundTimestamps) {
  if (currentRoundStartTimestamp === 0n) { // guard 1: not initialized
    runtime.log(`${feedId} / ${interval}s not initialized, skipping`);
    continue;
  }

  const roundEndTimestamp = currentRoundStartTimestamp + BigInt(interval);
  if (nowSeconds < roundEndTimestamp) { // guard 2: not ended yet
    runtime.log(`${feedId} / ${interval}s round not ended yet, skipping`);
    continue;
  }
}

```

When submitting the transaction, `_processReport` calls `_closeRound`, which calls `_resolveQuestion`, which calls `CTF.reportPayouts` on an already-resolved condition:

```

require(payoutDenominator[conditionId] == 0, "payout denominator already set");

```

This reverts on every attempt, indefinitely. Only `enableRoundConfig` (admin) recovers — but the M-5 issue can brick that path at historically-used boundaries, composing to a permanent lock.

Impact: The stuck disabled config poisons the entire CRE batch. `onCronTrigger.ts` collects all ready rounds across every configured `feedId/interval` pair and submits them in a single on-chain call via `writeCloseRounds`. Since there is no per-report error isolation in `_processReport` (line 245 of `ChainlinkUpDownAdapter.sol`) — the whole batch reverts — every invocation reverts, blocking all valid rounds across all other feeds from ever being settled. User funds in every active market are frozen indefinitely until the admin manually calls `enableRoundConfig` to recover the stuck config.

Direct violation of client concern 1 ("rounds are never stuck").

Recommended Mitigation: Advance `currentRoundStartTimestamp` to 0 when the last pre-created round closes with `isEnabled = false`. Setting `currentRoundStartTimestamp = 0` causes the CRE's existing guard (`if (currentRoundStartTimestamp === 0n) continue`) to skip this pair on every subsequent tick without any offchain changes. Alternatively, modify the offchain infrastructure to read and respect `isEnabled`.

Predict.fun: Fixed in commit [c608ae](#)

Cyfrin: Verified.

7.1.5 `enableRoundConfig` uses `<=` instead of `<`, forcing an unnecessary one-interval gap when re-enabling a round series

Description: `enableRoundConfig` is called by the admin to restart a round series after it has been disabled. It validates that the new series does not overlap with the previous one by checking that `newCurrentRoundStartTimestamp` is past the end of the previous series' last round:

```
if (newCurrentRoundStartTimestamp <= roundConfig.latestRoundStartTimestamp + interval) {
    revert ChainlinkUpDownAdapter__NewCurrentRoundStartTimestampMustBeGreaterThanLatestRoundEndTimestamp();
}
```

The expression `latestRoundStartTimestamp + interval` is the end timestamp of the last round in the previous series — as confirmed by the revert error name itself (`...GreaterThanLatestRoundEndTimestamp`). Using `<=` means that starting at exactly that boundary is rejected, even though the previous round has already fully ended at that timestamp.

Because `newCurrentRoundStartTimestamp` must also be divisible by `interval`, the smallest valid timestamp after the `<=` rejection is `latestRoundStartTimestamp + 2 * interval` — one full interval beyond the natural restart point.

We can also see that when rounds are created they start exactly at `latestRoundStartTimestamp + interval`

```
if (roundConfig.isEnabled) {
    uint32 newLatestTimestamp = roundConfig.latestRoundStartTimestamp + interval;
    roundConfig.latestRoundStartTimestamp = newLatestTimestamp;
    _createRound(feedId, newLatestTimestamp, interval);
}
```

Impact: Admins cannot re-enable a round series at the natural boundary immediately following the last round's end. The earliest valid restart is always at least one full interval later than expected. For long intervals (e.g. 1 hour, 4 hours, 1 day), this creates a mandatory dead period during which no market exists for that feed, directly harming continuity of the prediction market and user experience. A gap of one interval with no active market means no trading, no liquidity, and potential loss of user engagement for a predictable and avoidable window.

Proof of Concept: Setup:

- `interval = 900` (15 minutes)
- `latestRoundStartTimestamp = 54000` (15:00)
- Last round ends at: `54000 + 900 = 54900` (15:15)
- Admin calls `enableRoundConfig` with `newCurrentRoundStartTimestamp = 54900` (15:15 — the exact natural restart point):

`54900 <= 54900 → TRUE → REVERT`

Recommended Mitigation: Change `<=` to `<` so that the admin can restart the series at exactly the natural boundary.

Predict.fun: Fixed in commit [98569bc](#).

Cyfrin: Verified.

7.2 Low Risk

7.2.1 `setExpectedAuthor(address(0))` with non-empty workflow name causes onReport DoS

Description: Order-independent setters on `ChainlinkReceiverBase` skip the "name requires author" invariant at config time. One admin mis-step (setting author to zero while name is non-empty via `setExpectedAuthor(address(0))`) bricks settlement via the runtime check at `ChainlinkReceiverBase.sol:112`.

Impact: A single admin misconfiguration triggers a full onReport DoS — settlement is blocked until the admin reverses the change.

Recommended Mitigation: In `setExpectedAuthor`, revert if `_author == address(0) && s_expectedWorkflowName != bytes10(0)`. Fails fast at config time instead of at every onReport.

Predict.fun: Fixed in commit [a0aa366](#).

Cyfrin: Verified.

7.2.2 `ChainlinkUpDownAdapter::emergencyCloseRounds` accepts empty price array and negative prices

Description: `ChainlinkUpDownAdapter::emergencyCloseRounds` is gated by `EMERGENCY_CLOSE_ROUND_ROLE` but has two input validation gaps that can cause incorrect or silent settlement during an emergency:

1. Empty `endPrices[]` array is a silent no-op

```
for (uint32 i = 0; i < endPrices.length; i++) { ... }
```

Passing an empty array causes the loop body to never execute. The function returns with no revert, no event, and no rounds closed. An operator under time pressure may not notice.

2. Negative `endPrices[i]` forces deterministic Down outcome

```
_closeRound checks endPrice == 0 but not endPrice < 0:  
  
if (endPrice == 0) revert  
ChainlinkUpDownAdapter__EndPriceCannotBeZero();  
// no guard on negative values
```

A negative `endPrice` paired with any positive stored `startPrice` evaluates `startPrice > endPrice` → Down, regardless of actual market movement. This flips every round in the batch to Down via an admin typo.

The same gap exists in `_closeRound` itself; there is no `startPrice > 0` guard either. This is currently benign for BTC/ETH (Data Streams V3 cannot emit negative prices for spot feeds), but becomes load-bearing once rate or index feeds are added to the roster, since `int192` supports negative values and rate feeds can legitimately go negative.

Because only the role-holder can trigger this, the scenario could be a honest admin typo during a high-pressure emergency, not unprivileged exploitation.

Impact:

- Empty `endPrices[]`: silent failure - operator believes rounds are settled, they are not
- Negative `endPrice`: all rounds in the batch settle as Down regardless of actual price movement

Recommended Mitigation: Add guards at the top of `emergencyCloseRounds` and strengthen `_closeRound`:

```
function emergencyCloseRounds(  
    bytes32 feedId,  
    uint32 interval,  
    uint32 startTimestamp,  
    int192[] calldata endPrices  
) external onlyRole(EMERGENCY_CLOSE_ROUND_ROLE) whenPaused {  
    if (endPrices.length == 0) revert  
    ChainlinkUpDownAdapter__NoReportsToProcess();  
}
```

```

    for (uint32 i = 0; i < endPrices.length; i++) {
        if (endPrices[i] <= 0) revert
ChainlinkUpDownAdapter__EndPriceCannotBeZeroOrNegative();
        uint32 roundStartTimestamp = startTimestamp + i * interval;
        _closeRound(feedId, interval, roundStartTimestamp,
endPrices[i]);
        emit ChainlinkUpDownAdapter__RoundEmergencyClosed(feedId,
interval, roundStartTimestamp, endPrices[i]);
    }
}

```

Also harden `_closeRound` to reject non-positive prices from both the emergency and CRE paths:

```

if (endPrice <= 0) revert
ChainlinkUpDownAdapter__EndPriceCannotBeZeroOrNegative();

```

Predict.fun: Fixed in commit [478f88](#) & [6f0258](#).

Cyfrin: Verified.

7.2.3 Missing runtime input validation in admin setters and extension paths

Description: Grouping of runtime missing-input-validation findings in admin setters and extension paths on `ChainlinkUpDownAdapter` and `ChainlinkReceiverBase`. All are admin-gated but each lets an admin typo or misconfiguration produce a runtime fault the contract should have rejected at function entry.

1. `ChainlinkUpDownAdapter::initialize`, `extend`, `enableRoundConfig` **accept unbounded** `roundCount`

`roundCount` is unbounded in all three functions. An admin or extender typo causes out-of-gas on the creation transaction. Role-gated and bounded in practice, but the contract offers no defensive ceiling.

Impact: Out-of-gas on the creation transaction; transaction wasted, no state corruption.

Recommended: `if (roundCount > MAX_ROUND_COUNT) revert ...;` with `MAX_ROUND_COUNT` chosen such that `gas x MAX_ROUND_COUNT` stays under ~80% of the BSC block gas limit.

2. `ChainlinkReceiverBase::setIsWorkflowIdValid(bytes32(0), true)` **accepted**

An admin typo or badly-encoded metadata could enable a zero `workflowId`, opening the gate for any caller whose metadata encodes a zero workflow id.

Impact: Admin misconfiguration could disable workflow-id validation entirely, allowing any caller metadata encoding a zero workflow id to pass the gate.

Recommended: Reject `workflowId == bytes32(0)` in `setIsWorkflowIdValid`:

```

if (workflowId == bytes32(0)) revert InvalidWorkflowId();

```

Predict.fun: Fixed in commit [981b4a4](#).

Cyfrin: Verified.

7.2.4 Missing CRE-side runtime validation of config values before use

Description: Grouping of off-chain / CRE-side defensive-depth gaps where TypeScript workflow code, init scripts, or config files pass values into critical operations (settlement calls, chain-scoped deploy actions) without validating that those values are within acceptable bounds at runtime. In every case the consequence is operator-footgun: a misconfiguration silently produces wrong behaviour rather than failing loudly.

1. CRE runtime.config lacks runtime validation

The CRE TypeScript workflow defines a Config type in types.ts that provides compile-time structural typing for config fields (dataStreamsEndpoint, adapterAddress, intervals[], gasLimit). TypeScript types are erased at runtime and do not validate the actual JSON values loaded from config/*.json during execution. No runtime schema validation, semantic boundary checks, or domain-pinning assertions are applied before these values flow into critical operations.

Impact: An invalid config value silently passes type-check at build time, then produces runtime failures (dead calls, reverts, silent settlement stalls) that are hard to diagnose because no input-validation layer exists to reject the bad value at the boundary.

Recommended: Add a runtime schema validation layer (e.g. zod, manual assertions) that rejects invalid values before they are used. Check: non-zero adapterAddress, non-empty valid-URL dataStreamsEndpoint, positive intervals[] divisible by 60, non-zero parseable gasLimit.

2. CRE workflow does not validate adapterAddress != 0x0 before submitting reports

The CRE workflow reads adapterAddress from its config file (config.production.json or config.staging.json) and uses it as the target for Multicall3.aggregate3 (in readRoundConfigs.ts) and evmClient.writeReport (in writeCloseRounds.ts). No validation against the zero address before these calls. If adapterAddress is the zero address (as the shipped config.production.json currently is, pending mainnet deployment), the staticcall to BSC returns (success=true, data=0x), viem's decodeFunctionResult throws AbiDecodingZeroDataError on empty bytes, and onCronTrigger fails silently - the whole cron tick throws. Rounds never close and the operator sees no visible error.

Impact: If adapterAddress is misconfigured to zero at deployment, the CRE workflow silently halts settlement indefinitely. All rounds stay stuck until the config is fixed and the workflow restarted.

Recommended: Add a validation check in readRoundConfigs.ts or as an early-exit guard in onCronTrigger.ts:

```
if (config.adapterAddress === "0x00000000000000000000000000000000" ||
    !isAddress(config.adapterAddress)) {
  throw new Error(`Invalid adapterAddress in config: ${config.adapterAddress}`);
}
```

3. Init script hardcodes testnet adapter address but accepts mainnet chainid

ChainlinkUpDownAdapterInitializeRoundSeries.s.sol:33 hardcodes the adapter address as 0xDB923974731Bdf5C4b1c046522Bf7F99Ee467257 (BSC testnet). Yet _deployerPrivateKey() in the same script accepts block.chainid == 56 (BSC mainnet) and loads BSC_MAINNET_KEY. If an operator runs forge script --chain-id 56 intending mainnet execution, the script signs a transaction with the mainnet private key targeting the testnet address on mainnet - either a no-code address (revert, wasted BNB at mainnet gas prices) or a coincidentally-deployed unrelated contract (unintended behaviour on an unrelated contract).

Impact: Operator footgun with real financial consequence. The script as written accepts the mainnet chainid and has no defense against this misconfiguration.

Recommended: Either make the adapter address chain-branched (like deployment parameters in ChainlinkUpDownAdapterDeployment.s.sol), or add a require(block.chainid == expectedChainId, ...) assertion at the top of the script that matches the hardcoded adapter address.

Predict.fun: Fixed in commit [54b521](#), [ba429db](#), [1209b33](#), [2371d48](#)

Cyfrin: Verified.

7.2.5 Inconsistent expiry comparison between CRE pre-flight and onchain validation

Description: The CRE off-chain workflow and the onchain adapter apply asymmetric expiry checks on Data Streams reports.

The CRE check in `fetchDataStreamsReportsBulk.ts:93-95` uses strict less-than:

```
if (decodedData.expiresAt < nowSeconds) {  
  throw new Error(`Report expired for ${report.feedID}...`);  
}
```

This **accepts** a report when `expiresAt == nowSeconds`. Onchain, `ChainlinkUpDownAdapter.sol:354` uses greater-than-or-equal:

```
if (block.timestamp >= expiresAt) {  
  revert ChainlinkAdapter__ReportExpired();  
}
```

This **rejects** when `block.timestamp == expiresAt`.

Between the CRE DON validating a report and the resulting transaction being mined on BSC (~3 second blocks + Keystone Forwarder pipeline latency 6-15 seconds), `block.timestamp` advances beyond the CRE's `nowSeconds`. Any report where `expiresAt - nowSeconds < pipeline_latency` passes CRE validation but reverts onchain.

These are semantically mismatched.

Impact: In practice this is harmless (the tx will revert) and Chainlink DON sets `expiresAt` hours/days ahead of `observationsTimestamp`, so the 1-second boundary gap is unreachable. The on-chain check remains authoritative regardless.

Recommended Mitigation: Align CRE expiry check with on-chain semantics (`<=` instead of `<`) for consistency.

```
//fetchDataStreamsReportsBulk.ts:94  
  
if (decodedData.expiresAt <= nowSeconds) {  
  throw new Error(`Report expired for ${report.feedID}: expiresAt=${decodedData.expiresAt},  
    ↳ now=${nowSeconds}`);  
}
```

Predict.fun: Fixed in commit [53138ab](#).

Cyfrin: Verified.

7.2.6 enableRoundConfig skips _validateInterval check allowing re-activation for invalidated intervals

Description: Both `initialize()` (`ChainlinkUpDownAdapter.sol:70`) and `extend()` (`ChainlinkUpDownAdapter.sol:86`) call `_validateInterval(interval)` before operating which enforces the `isIntervalValid` allowlist. `enableRoundConfig()` (`ChainlinkUpDownAdapter.sol:155`) does not. An admin can therefore call `enableRoundConfig` for an interval that was explicitly invalidated via `updateIsIntervalValid(interval, false)` bypassing the interval governance policy.

Once re-enabled, the series runs normally: the CRE delivers reports, `_closeRound` resolves markets and rolls forward indefinitely (rolling creation in `_closeRound` also does not check `isIntervalValid` consistent with M-4 design. The disallowed interval remains operationally active with no further admin action required to sustain it.

Example:

```
// Setup - interval valid, series running normally  
updateIsIntervalValid(300, true)  
initialize(FEED_A, 300, T0, price, 3) // creates rounds at T0, T0+300, T0+600  
  
// Policy change - deprecate 5-min interval  
disableRoundConfig(FEED_A, 300) // stops rolling creation  
// CRE winds down: closes rounds at T0, T0+300, T0+600
```

```

updateIsIntervalValid(300, false)           // interval now policy-disallowed

// Verify gates work for lesser roles:
extend(FEED_A, 300, 3)                     // REVERTS: InvalidInterval ✓
initialize(FEED_B, 300, T1, price, 3)      // REVERTS: InvalidInterval ✓

// But admin can still bypass:
enableRoundConfig(FEED_A, 300, T1, price, 3) // SUCCEEDS - no _validateInterval call
// FEED_A/300 series now live again on a disallowed interval
// _closeRound rolling creation sustains it indefinitely

```

Impact: DEFAULT_ADMIN_ROLE can silently re-activate a market series for a disallowed interval without first re-enabling the interval. In a multi-admin setup, Admin A may invalidate an interval as a policy decision ("this interval is deprecated"), while Admin B calls enableRoundConfig without realizing the interval is disallowed.

So the check that should surface the conflict is absent. The active series then continues indefinitely on a policy-disallowed interval via _closeRound rolling creation.

Proof of Concept:

```

1. Admin: `updateIsIntervalValid(300, true)`
2. Initializer: `initialize(FEED, 300, T0, price, 3)`
3. Admin: `disableRoundConfig(FEED, 300)`
4. CRE closes all 3 pre-created rounds (wind-down complete)
5. Admin: `updateIsIntervalValid(300, false)` // interval now invalid
6. Extender: `extend(FEED, 300, 3)`           → REVERTS with InvalidInterval ✓
7. Initializer: `initialize(FEED2, 300, ...)` → REVERTS with InvalidInterval ✓

8. Admin: `enableRoundConfig(FEED, 300, newTs, price, 3)` → SUCCEEDS × (no interval check)
// Series now live on disallowed interval; rolling creation sustains indefinitely

```

Note: _validateRoundConfigInitialized (L342) does not prevent this, it only checks that both timestamps are non-zero, which they are (set during the original initialize() call and unchanged by disableRoundConfig).

Recommended Mitigation: Add _validateInterval(interval) at the top of enableRoundConfig, consistent with initialize() (L70) and extend() (L86):

```

function enableRoundConfig(
    bytes32 feedId,
    uint32 interval,
    uint32 newCurrentRoundStartTimestamp,
    int192 startPrice,
    uint256 roundCount
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    _validateInterval(interval); // ADD THIS
    RoundConfig storage roundConfig = _getCurrentRoundConfig(feedId, interval);
    // ...
}

```

Predict.fun: Fixed in commit [0065ac](#).

Cyfrin: Verified.

7.3 Informational

7.3.1 `chainlinkReports[i].interval` is caller-controlled, not in the DON-signed payload

Description: `ReportV3` (defined in `IChainlinkAdapter.sol:13-23`) contains `{feedId, validFromTimestamp, observationsTimestamp, nativeFee, linkFee, expiresAt, price, bid, ask}` — there is **no** `interval` field. `ChainlinkUpDownAdapter::_processReport` at `ChainlinkUpDownAdapter.sol:244-245` reads `interval = chainlinkReports[i].interval` from the OUTER `ChainlinkReport[]` wrapper (set by the caller / CRE / forwarder), then calls `_closeRound(report.feedId, interval, report.observationsTimestamp - interval, report.price)`. A malicious or mis-configured forwarder can pair a genuinely DON-signed `ReportV3` with an attacker-chosen `interval`, routing a valid price to the wrong round.

`IChainlinkAdapter.sol:13-23` shows the `ReportV3` struct with no `interval` field. `ChainlinkUpDownAdapter.sol:244-245` reads `interval` from the outer wrapper, not from a signed field.

Impact: When multiple `intervals` are active on the same `feedId` and the alternative `startTimestamp` is also current (e.g. both `ts % 900 == 0` && `ts % 300 == 0`), outcome steering becomes possible. Otherwise the mismatched route reverts via `NotCurrentRound`, limiting likelihood to Medium.

Recommended Mitigation: Either (a) move `interval` into the DON-signed payload so it cannot be tampered with, or (b) add a local consistency check: derive the `interval` from the target round config and require the caller-supplied value to match. Simplest local fix: ignore `chainlinkReports[i].interval` entirely and instead look up the active (`feedId, interval`) pair for which `observationsTimestamp - interval == currentRoundStartTimestamp` — resolve `interval` from adapter state, not from caller input.

Predict.fun: Acknowledged, this will revert since it can not match with any round config and the current round start timestamp.

7.3.2 `ChainlinkAdapter::_constructPayouts` silent `[0,0]` fallback on unexpected outcome

Description: `ChainlinkAdapter::_constructPayouts` uses `if / else if / else if` without a default. An unexpected price input silently returns `[0,0]` -> CTF locks funds (no payout for anyone). Not reachable today.

Impact: If a future refactor introduces an unhandled outcome path, funds lock in CTF with no payout.

Recommended Mitigation: Add an explicit `else revert ChainlinkAdapter__InvalidOutcome(price);` default branch.

Predict.fun: Fixed in commit [54942ac](#).

Cyfrin: Verified.

7.3.3 Missing `whenNotPaused` on `onReport` pre-validation and no reentrancy guards on CTF calls

Description: Pause semantics are inconsistent in `ChainlinkReceiverBase::onReport` — metadata validation runs even when paused (though `_processReport` itself is guarded by `whenNotPaused`). CTF and `VerifierProxy` are trusted integrations, so the reentrancy surface is bounded.

Impact: During pause, metadata validation still runs (gas waste) before the inner `whenNotPaused` reverts. No direct fund impact because CTF / `VerifierProxy` are trusted, but defensive reentrancy coverage is missing.

Recommended Mitigation: Move `whenNotPaused` to the top of `onReport` so pre-validation short-circuits under pause. Add OZ `ReentrancyGuard` on `onReport` defensively.

Predict.fun: Acknowledged; `ChainlinkReceiverBase` is a copy of Chainlink's `ReceiverTemplate` so we tried to make as few changes to the base contract as possible. As there are no demonstrated security issues we'd prefer not to make any additional changes to this contract.

7.3.4 Event emission gaps and telemetry asymmetries

Description: `ChainlinkUpDownAdapter::_closeRound` normal path emits no close event — only emergency-`CloseRounds` does. `RoundStartPriceSet` emits the recorded (possibly stale — see the H-1 issue) price rather than the supplied one. Off-chain reconstruction harder.

Impact: Off-chain indexers and monitoring lose visibility into normal-path closures and may log misleading start-price values.

Recommended Mitigation: Emit `ChainlinkUpDownAdapter__RoundClosed(feedId, interval, startTimestamp, endPrice, outcome)` at the tail of `_closeRound`. Fix `RoundStartPriceSet` to emit the supplied `startPrice` (see the H-1 issue's mitigation).

Predict.fun: Acknowledged, they conclude that having the `RoundStartPriceSet` event is good enough.

7.3.5 Do not initialize local variables to their default values

Description: Solidity locals default to zero. Explicit `= 0` in declarations / for-loop initializers wastes gas in some solidity versions and is redundant code in others:

```
ChainlinkUpDownAdapter.sol
120:     for (uint32 i = 0; i < endPrices.length; i++) {
228:     for (uint256 i = 0; i < chainlinkReports.length; i++) {
241:     for (uint256 i = 0; i < chainlinkReports.length; i++) {

ChainlinkReceiverBase.sol
167:     for (uint256 i = 0; i < 10; i++) {
192:     for (uint256 i = 0; i < data.length; i++) {
```

Impact: Minor per-call gas waste in some solidity versions and redundant code in others.

Recommended Mitigation: Drop the `= 0` — use `for (uint256 i; i < len; ) {...}`. Line 395 already follows this style; apply everywhere.

Predict.fun: Acknowledged, we ran a snapshot diff & it didn't change much.

7.3.6 Deployer EOA retains `DEFAULT_ADMIN_ROLE` after deployment

Description: `ChainlinkReceiverBase`'s constructor grants `DEFAULT_ADMIN_ROLE` to `msg.sender` (the deployer EOA, loaded from the `BSC_MAINNET_KEY` / `BSC_TESTNET_KEY` env var). `ChainlinkUpDownAdapterDeployment.s.sol` then grants the same role to the configured multisig but does NOT revoke the deployer's role. After deployment, two principals hold `DEFAULT_ADMIN_ROLE`: the multisig (intended) and the deployer EOA (unintended).

The protocol's documented trust model places administrative authority with the multisig. A second admin principal that isn't the multisig expands the actual trust surface beyond the documented model — the adapter can be fully administered by whoever controls the deployer key, independent of the multisig's state.

Impact: Any action `DEFAULT_ADMIN_ROLE` can perform (granting `PAUSER_ROLE` / `EMERGENCY_CLOSE_ROUND_ROLE` to new addresses, rotating the forwarder address, changing the expected author/workflow name, enabling/disabling round configs) is available to the deployer key as well as the multisig. The practical centralization footprint depends on how the deployer key is managed post-deploy. If the deployer is a hot CI key or a developer workstation, the protocol's effective trust model is weaker than the multisig-only model documented.

Recommended Mitigation: In `ChainlinkUpDownAdapterDeployment.s.sol`, after granting `DEFAULT_ADMIN_ROLE` to the multisig and the other roles to their holders, revoke the deployer's role in the same transaction:

```
// After all grantRole calls at the end of run():
chainlinkUpDownAdapter.renounceRole(
    chainlinkUpDownAdapter.DEFAULT_ADMIN_ROLE(),
    vm.addr(deployerPrivateKey)
);
```

Verify post-deploy via `hasRole(DEFAULT_ADMIN_ROLE, deployer)` that the deployer no longer holds admin and the multisig is the sole admin principal.

Predict.fun: Acknowledged; typically we use the multisig to revoke the role of the deployer key so that we can be sure the multisig has the default admin role. Otherwise, we can end up with a contract with no default admin.

7.3.7 Pre-deploy placeholders and deploy-script gaps: unset roles, placeholder addresses and stale timestamps

Description: Grouping of pre-deployment configuration gaps - placeholder values, unfilled TODOs, and missing role grants in deploy scripts. Each sub-item is a distinct instance. All are non-exploitable (pre-deploy or role-gated) but each represents a runbook step that must happen before mainnet or the protocol misbehaves.

1. Mainnet deploy branch grants EXTENDER_ROLE to address(0) - TODO placeholder

ChainlinkUpDownAdapterDeployment.s.sol:48 passes extender: 0x0000000000000000000000000000000000 for the BSC mainnet branch, with an inline `// TODO: operator address comment`. If the script runs as-is on mainnet, EXTENDER_ROLE is granted to address(0), meaning no real address holds the role until an admin grants it post-deploy.

Impact: `extend()` is uncallable on mainnet until an admin manually grants EXTENDER_ROLE to a real operator address. Recoverable misconfiguration, not a vulnerability.

Recommended: Fill in the real operator address in the mainnet branch before running the deploy script. Add defensive requires at the top of `run()`:

```
require(params.extender != address(0), "extender address not set");
require(params.initializer != address(0), "initializer address not set");
require(params.msgsig != address(0), "msig address not set");
```

2. config.production.json ships with zero adapterAddress - pre-deploy placeholder

cre/data-stream-resolution/close-rounds/config.production.json line 4 sets "adapterAddress": "0x0000000000000000000000000000000000". Pre-deployment placeholder to be updated once the adapter is deployed to BSC mainnet.

Impact: If the config reaches CRE production with the zero address in place, the CRE workflow silently no-ops on every cron tick rather than loudly failing. The CRE workflow itself does not validate `adapterAddress != 0x0` before submitting reports, so the misconfiguration is not caught on the CRE side either - tracked as a separate Low finding in this repo.

Recommended: Add "update adapterAddress in config.production.json" as a named step in the production deployment runbook. Consider a commit-time CI check that rejects config files containing the zero address.

3. Init script hardcodes past startTime - pre-deploy placeholder needs explicit update step

ChainlinkUpDownAdapterInitializeRoundSeries.s.sol:38 hardcodes `uint32 startTime = 1776198600` (Tue Apr 14 2026 20:30 UTC). If the script runs meaningfully after this timestamp, every pre-created round has an `endTimeStamp` in the past - outcomes are deterministically known from the Chainlink Data Streams historical API before any CRE run. No inline `// TODO` comment flags this to operators (unlike the `extender` placeholder in the deploy script).

Impact: If run verbatim without updating `startTime`, on-chain state ends up with pre-created rounds whose outcomes are publicly knowable before the CRE settles them. The related contract-level defensive-depth gap (missing sanity bound in `_initialize`, tracked as a separate Informational finding) would catch this even if operator discipline fails.

Recommended: Add an inline comment + runtime assertion:

```
// TODO: update startTime to a recent timestamp (within the last interval) before running
uint32 public constant startTime = 1776198600;
// ...
require(
    block.timestamp - startTime < uint256(5 minutes),
    "startTime stale; update before running"
```



```
);
```

Alternatively, derive `startTime` at script runtime:

```
uint32 startTime = uint32(block.timestamp) / interval * interval - interval;
```

4. PAUSER_ROLE and EMERGENCY_CLOSE_ROUND_ROLE never granted in deploy script

`ChainlinkAdapter::togglePaused` and `ChainlinkUpDownAdapter::emergencyCloseRounds` have no role-holder post-deploy. Legitimate emergency recovery (DS API outage, expired reports, stuck rounds) cannot happen until admin remembers to self-grant.

Impact: Delayed emergency recovery.

Recommended: Grant `PAUSER_ROLE` and `EMERGENCY_CLOSE_ROUND_ROLE` explicitly to the `msig` inside `ChainlinkUpDownAdapterDeployment.s.sol`. Emit an alert if any role is granted from a non-deploy-script tx post-deploy.

Predict.fun: Fixed in commit [333d76a](#).

Cyfrin: Verified.

7.3.8 Documentation issues: NatSpec/code mismatches, missing `@inheritdoc` and missing clarifying comments

Description: Grouping of documentation and comment issues: NatSpec that disagrees with code, inconsistent doc styles, and missing clarifying comments on non-obvious encoding or external-call semantics.

1. Missing `@inheritdoc` and NatSpec inconsistencies

`ChainlinkUpDownAdapter` mixes `@inheritdoc` and inline NatSpec. `ChainlinkAdapter::togglePaused` NatSpec (line 55) says `DEFAULT_ADMIN_ROLE` but the modifier (line 57) is `PAUSER_ROLE` - a direct doc/code mismatch.

Recommended: Fix the `DEFAULT_ADMIN_ROLE` → `PAUSER_ROLE` doc in `togglePaused`. Decide on one NatSpec style (`@inheritdoc` where interface exists, inline otherwise) and apply consistently.

2. `conditionId` uses `abi.encodePacked` vs `abi.encode` elsewhere - add comment

`ChainlinkUpDownAdapter.sol:175` uses `abi.encodePacked(address(this), latestQuestionID, uint256(2))` because it must match CTF's `getConditionId`. Other key derivations in the codebase use `abi.encode`. The inconsistency is correct but confusing.

Recommended: Add a clarifying comment:

```
// abi.encodePacked matches CTF.getConditionId(address,bytes32,uint256) layout.
bytes32 conditionId = keccak256(abi.encodePacked(address(this), latestQuestionID, uint256(2)));
```

3. Metadata length NatSpec comment mismatch (62 vs 64 bytes)

NatSpec in `ChainlinkReceiverBase` mentions 62 bytes in some places and 64 in others. Benign today (adapter reads fixed offsets) but a maintainability hazard.

Recommended: Unify to the actual `abi.encodePacked(bytes32, bytes10, address) = 62 bytes`.

4. CTF.prepareCondition and CTF.reportPayouts return values discarded - document expectations

CTF.prepareCondition(questionID, 2) at ChainlinkUpDownAdapter.sol:304 and CTF.reportPayouts(questionID, payouts) at ChainlinkAdapter.sol:76 are called without inline documentation of their revert semantics (Gnosis CTF prepareCondition reverts on duplicate; reportPayouts reverts on re-report).

Recommended: Add inline comments referencing the CTF behavior:

```
// reverts if already prepared (duplicate questionID)
CTF.prepareCondition(questionID, 2);
```

Predict.fun: Fixed in commit [4ae46fc](#).

Cyfrin: Verified.

7.3.9 Code style and naming conventions: inconsistent loops, storage prefixes, magic numbers and conflicting names

Description: Grouping of code-style and naming-convention inconsistencies. All are non-behavioral.

1. Loop style inconsistent between ChainlinkUpDownAdapter::extend and _initialize

extend uses for (uint32 i = 1; i <= roundCount; i++) { ts += interval; ... } (inclusive bound from 1). _initialize uses for (uint32 i; i < roundCount; i++) { ... } (exclusive bound from 0). Both create roundCount rounds; the mixed style is confusing and easy to miscount when refactoring either loop.

Recommended: Standardize on i; i < roundCount; ++i everywhere.

2. State variable naming - s_ prefix inconsistency

ChainlinkReceiverBase uses s_ for private storage (s_forwarderAddress, s_expectedAuthor, s_expectedWorkflowName), but isWorkflowIdValid in the same contract does not. ChainlinkUpDownAdapter uses no prefix at all.

Recommended: Pick one convention and apply across the hierarchy - either drop s_ from ChainlinkReceiverBase or adopt it everywhere.

3. Magic numbers without named constants

Several magic numbers appear inline: 60 (seconds per minute), 2 (outcome slot count), 3 (DS report version), 1 ether / 0.5 ether / 0 outcome discriminators, 62 (metadata length).

Recommended: Replace with named constants:

```
uint32 private constant MIN_INTERVAL_SECONDS = 60;
uint8  private constant OUTCOME_SLOT_COUNT  = 2;
uint16 private constant CHAINLINK_REPORT_V3 = 3;
int256 private constant OUTCOME_UP          = 1 ether;
int256 private constant OUTCOME_FLAT        = 0.5 ether;
int256 private constant OUTCOME_DOWN       = 0;
uint256 private constant MIN_METADATA_LENGTH = 62;
```

4. ChainlinkUpDownAdapter::initialize function name collides with upgradeable-proxy convention

ChainlinkUpDownAdapter::initialize is the round-series initializer, NOT the upgradeable-proxy Initializable.initialize. The contract is not upgradeable. The naming collision confuses integrators and static-analysis tooling that treats initialize as the proxy initializer.

Recommended: Rename to `initializeRoundSeries` or `createRoundSeries`.

Predict.fun: Fixed in commit [46ab07e](#).

Cyfrin: Verified.

7.3.10 Missing events: constructor init event absent and admin-mutating events lack `msg.sender`

Description: Grouping of events-related observability gaps. Each sub-item is a distinct instance.

1. Admin-mutating events lack `msg.sender` attribution

`IsWorkflowIdValidUpdated`, `ForwarderAddressUpdated`, and other admin-mutating events in `ChainlinkReceiverBase` do not emit the caller. Audit trail limited.

Recommended: Add address indexed caller (= `msg.sender`) to every admin-mutating event.

2. Constructor does not emit an initialization event

`ChainlinkAdapter` constructor does not emit an event recording `VERIFIER_PROXY` or `CTF`. Off-chain indexers cannot identify these without reading storage or parsing constructor args.

Recommended: Emit `ChainlinkAdapterInitialized(address verifierProxy, address ctf)` in the constructor.

Predict.fun: Fixed in commit [4ba7df](#).

Cyfrin: Verified.

7.3.11 Missing deploy-time input validation in constructor and initializer

Description: Grouping of deploy-time defensive-depth gaps where constructor or initializer inputs are stored or consumed without a sanity check. Both members are Informational per the Gate 2 deploy-time exception - a bad value produces an obviously-broken state the operator can recover from by redeployment, but in each case the contract offers no defensive reject-on-bad-input that would convert an operator mistake into a clean revert instead of silent dysfunction.

1. `ChainlinkAdapter` constructor missing zero-address validation on `_verifierProxy` and `_ctf`

`ChainlinkAdapter` constructor assigns both `_verifierProxy` and `_ctf` as immutable without zero-address checks. Deploy-time misconfiguration is irrecoverable because the fields are immutable.

```
40:     constructor(address _forwarderAddress, address _verifierProxy, address _ctf)
    ↪ ChainlinkReceiverBase(_forwarderAddress) {
45:         VERIFIER_PROXY = IVerifierProxy(_verifierProxy);
46:         CTF = IConditionalTokens(_ctf);
47:     }
```

Impact: Zero-address footgun at deploy; irrecoverable because fields are immutable.

Recommended:

```
if (_verifierProxy == address(0)) revert ChainlinkAdapter__InvalidVerifierProxy();
if (_ctf == address(0)) revert ChainlinkAdapter__InvalidCTF();
```

2. ChainlinkUpDownAdapter::_initialize lacks lower-bound sanity check on startTimestamp

_initialize enforces only startTimestamp < block.timestamp. There is no lower bound - any timestamp in the past is accepted, from five minutes ago to twenty-five years ago. The expected operational flow is that initialize is called with a startTimestamp within the last interval; an operator mistake (stale script, stale variable, copy-paste from prior deployment) could pass a deep-past startTimestamp without the contract noticing.

Impact: If startTimestamp is sufficiently far in the past, the contract pre-creates rounds whose endTimestamps are also in the past. Because CTF.prepareCondition has no tradability gate and the Data Streams historical API serves observations at those timestamps, every pre-created round has a deterministic outcome that any public observer of RoundSeriesInitialized can recover and trade on. Severity is Informational because only INITIALIZER_ROLE can call initialize - this is defensive-depth against operator mistakes rather than an unprivileged exploit.

Source: ChainlinkUpDownAdapter.sol:377-379.

Recommended:

```
if (block.timestamp - startTimestamp > interval * 2) {
    revert ChainlinkUpDownAdapter__StartTimestampTooOld();
}
```

The threshold should be tight enough to reject stale values but loose enough to tolerate reasonable propagation and scheduling windows.

Predict.fun: Acknowledged, the initializer will be a multi-sig so it takes time to gather all the signature, worst case they just redeploy.

7.3.12 emergencyCloseRounds propagates admin-supplied endPrice as next round's startPrice allowing outcome predetermination

Description: emergencyCloseRounds (ChainlinkUpDownAdapter.sol:114-126) accepts admin-supplied endPrices with no validation beyond endPrice != 0 (_closeRound:261). When _closeRound executes, it calls _setNextRoundStartPriceIfPresent (L289), which writes the endPrice as the next round's startPrice if that round is pre-created and its price slot is empty (L319: if (recordedStartPrice == 0)).

```
if (present) {
    int192 recordedStartPrice = prices[feedId][startTimestamp];
    if (recordedStartPrice == 0) {
        prices[feedId][startTimestamp] = startPrice;
        recordedStartPrice = startPrice;
    }
    emit ChainlinkUpDownAdapter__RoundStartPriceSet(feedId, interval, startTimestamp,
        ↪ recordedStartPrice);
}
```

This guard does **not** protect against extreme prices as it only prevents double-writes. The next round's startPrice is always 0 before the current round closes (it is set exclusively by the current round's close), so the guard always passes on first write. An admin can supply any extreme price and it propagates unchecked.

Attack Path:

```
State: BTC/USD running at ~$78,243
Round T0: startPrice = 78,243e18 (set at init)
Round T1: startPrice = 0, pre-created, CTF condition live
(UP/DOWN tokens for T1 already tradeable on secondary CTF market)

1. Attacker buys UP conditional tokens for Round T1 on secondary market
   before pausing - positions taken while contract is still live
```

```
2. Admin pauses contract (PAUSER_ROLE)

3. emergencyCloseRounds(FEED, 300, T0, [1])
   → T0 resolves: 78,243e18 vs 1 → DOWN
   → prices[FEED][T1] = 1 ← T1 startPrice now extreme low
(extreme startPrice visible onchain only AFTER this tx lands and attacker's positions already taken,
↪ outcome now locked)

4. Admin unpauses contract

5. CRE closes T1 with real Chainlink price (~78,243e18)
   → T1: startPrice=1 vs endPrice=78,243e18 → UP guaranteed
   → Attacker redeems full T1 pot
```

Impact: A holder of EMERGENCY_CLOSE_ROUND_ROLE (or a colluder) can predetermine the next round's outcome by supplying an extreme endPrice during emergency close.

The attack is not detectable in advance by regular users since the manipulated startPrice only becomes visible onchain after the emergency close transaction lands, at which point attacker positions are already taken and the outcome is locked. No remediation path exists for users once the extreme price is written.

It requires admin-level role collusion and contract must be in paused (emergency) state. Real user funds in the next round are at risk if role holders are also market participants making it low impact.

Proof of Concept:

```
1. Initialize round series with startPrice = 78,243e18
2. Pause contract
3. Emergency close round T0 with endPrice = 1 (1 wei)
4. prices[FEED][T0+300] = 1 (next round's startPrice)
5. CRE closes T0+300 with any real Chainlink price > 1 → outcome = UP guaranteed
```

Recommended Mitigation: Do not add a price delta cap, emergency close exists for abnormal conditions (flash crashes, stale feeds, black swan events) where large price moves are legitimate. A delta restriction would break valid emergency use.

Enforce operational separation at the role level:

Ensure EMERGENCY_CLOSE_ROUND_ROLE is held exclusively by a neutral multi-sig with no trading positions. This role must never be held by any entity that participates in the prediction markets, as the emergency price input directly determines subsequent round outcomes. Document this constraint explicitly in deployment runbooks and governance policies.

Alternatively, at the contract level: derive the next round's startPrice from an independent onchain oracle snapshot rather than propagating the admin-supplied endPrice if possible.

Predict.fun: Acknowledge, In the case of emergency, the current time would be X+10 already, so every round from X to X+9's time would already be known offchain, and those markets would be traded towards those outcomes offchain even if not already resolved onchain.

7.4 Gas Optimization

7.4.1 Cache storage array length

Description: Several loops in ChainlinkUpDownAdapter and ChainlinkReceiverBase read `.length` repeatedly; `_processReport` iterates over CRE batches every 30s, making the savings material.

```
ChainlinkUpDownAdapter.sol
228:     for (uint256 i = 0; i < chainlinkReports.length; i++) {
241:     for (uint256 i = 0; i < chainlinkReports.length; i++) {
```

Impact: Wasted gas per iteration on every CRE cron tick; compounds with batch size.

Recommended Mitigation:

```
uint256 len = chainlinkReports.length;
for (uint256 i; i < len; i++) {
    // body
}
```

Predict.fun: Fixed in commit [2ca1146](#).

Cyfrin: Verified.

7.4.2 Emit before state change or use input value to avoid storage re-read

Description: Several setters in ChainlinkReceiverBase cache a previous storage value into a local purely for an event, or re-read storage after writing it:

```
ChainlinkReceiverBase.sol
138:     address previousAuthor = s_expectedAuthor;
139:     s_expectedAuthor = _author;
140:     emit ExpectedAuthorUpdated(previousAuthor, _author);

170:     s_expectedWorkflowName = bytes10(first10);
171:     emit ExpectedWorkflowNameUpdated(previousName, s_expectedWorkflowName); // re-reads
↳ storage!

249:     address previousForwarder = s_forwarderAddress;
250:     s_forwarderAddress = _forwarder;
251:     emit ForwarderAddressUpdated(previousForwarder, _forwarder);
```

Line 171 re-reads `s_expectedWorkflowName` (warm SLOAD ~100 gas) immediately after writing — use the local instead.

Impact: Minor gas waste per admin setter call.

Recommended Mitigation:

```
function setExpectedAuthor(address _author) external onlyRole(DEFAULT_ADMIN_ROLE) {
    emit ExpectedAuthorUpdated(s_expectedAuthor, _author);
    s_expectedAuthor = _author;
}

bytes10 newName = bytes10(first10);
emit ExpectedWorkflowNameUpdated(s_expectedWorkflowName, newName);
s_expectedWorkflowName = newName;
```

Predict.fun: In commit [a0aa366](#) `s_expectedAuthor` and `s_expectedWorkflowName` were refactored into a mapping and now set in `setIsWorkflowAuthorAndNameValid`.

Cyfrin: After the mentioned workflow author & name refactoring, the only remaining optimization relevant to this issue is in `_setForwarderAddress` where `previousForwarder` can be optimized away by emitting the event first:

```

function _setForwarderAddress(address _forwarder) private {
    if (_forwarder == address(0)) {
        revert InvalidForwarderAddress();
    }
-   address previousForwarder = forwarderAddress;
+   emit ForwarderAddressUpdated(forwarderAddress, _forwarder);
    forwarderAddress = _forwarder;
-   emit ForwarderAddressUpdated(previousForwarder, _forwarder);
}

```

7.4.3 Cache roundConfigIndices[feedId][interval] inside _createRound loops

Description: ChainlinkUpDownAdapter::_createRound is called inside extend and _initialize loops (up to roundCount iterations). Each call reads roundConfigIndices[feedId][interval] via _getRound / roundKey. The index never changes within the loop; caching once outside avoids N-1 redundant warm SLOADs (~100 gas each).

```

ChainlinkUpDownAdapter.sol
103:     for (uint32 i = 1; i <= roundCount; i++) {
104:         ts += interval;
105:         _createRound(feedId, ts, interval);
106:     }
302:     function _createRound(bytes32 feedId, uint32 startTimestamp, uint32 interval) private {
305:         rounds[roundKey(feedId, interval, roundConfigIndices[feedId][interval], startTimestamp)] =
↪     questionID;

```

Impact: ~100 gas per extra loop iteration wasted on redundant warm SLOADs.

Recommended Mitigation:

```

uint256 idx = roundConfigIndices[feedId][interval];
for (uint32 i; i < roundCount; i++) {
    bytes32 questionID = keccak256(abi.encode(feedId, interval, ts));
    CTF.prepareCondition(questionID, 2);
    rounds[roundKey(feedId, interval, idx, ts)] = questionID;
    emit ChainlinkUpDownAdapter__RoundCreated(feedId, interval, ts, questionID);
    ts += interval;
}

```

Predict.fun: Acknowledged; we prefer the current version for readability.

7.4.4 ChainlinkUpDownAdapter::enableRoundConfig reads roundConfigIndices[feedId][interval] twice

Description: In ChainlinkUpDownAdapter::enableRoundConfig, the current index is read implicitly via _getCurrentRoundConfig (line 162) then explicitly again at line 180 (roundConfigIndices[feedId][interval] + 1). Both reads are on the success path.

```

ChainlinkUpDownAdapter.sol
162:     RoundConfig storage roundConfig = _getCurrentRoundConfig(feedId, interval);
180:     uint256 newRoundConfigIndex = roundConfigIndices[feedId][interval] + 1;
181:     roundConfigIndices[feedId][interval] = newRoundConfigIndex;

```

Impact: Duplicate warm SLOAD (~100 gas) on the success path.

Recommended Mitigation:

```

uint256 currentIndex = roundConfigIndices[feedId][interval];
RoundConfig storage roundConfig = roundConfigs[roundConfigKey(feedId, interval, currentIndex)];
// ...
uint256 newRoundConfigIndex = currentIndex + 1;

```

```
roundConfigIndices[feedId][interval] = newRoundConfigIndex;
```

Predict.fun: Acknowledged; we prefer the current version for readability.

7.4.5 Fail-fast: move cheap input checks before storage reads

Description: Several functions in ChainlinkUpDownAdapter perform storage reads before cheap calldata validations. Moving cheap checks first avoids SLOADs on the revert path.

```
ChainlinkUpDownAdapter.sol
85:     function extend(bytes32 feedId, uint32 interval, uint256 roundCount) external
    ↪ onlyRole(EXTENDER_ROLE) {
86:         _validateInterval(interval);    // SLOAD isIntervalValid
87:
88:         if (roundCount == 0) {           // cheap calldata check
89:             revert ChainlinkUpDownAdapter__RoundCountTooLow();
90:         }
```

Saves ~2,100 gas on the roundCount == 0 revert path. Similarly initialize delegates cheap checks (startTimestamp >= block.timestamp, roundCount < 2, startPrice == 0, alignment) after two SLOADs — reordering saves ~2,200 gas on revert.

Impact: Wasted SLOAD gas on revert paths.

Recommended Mitigation:

```
function extend(bytes32 feedId, uint32 interval, uint256 roundCount) external onlyRole(EXTENDER_ROLE) {
    if (roundCount == 0) revert ChainlinkUpDownAdapter__RoundCountTooLow();
    _validateInterval(interval);
    // ...
}
```

Predict.fun: Fixed in commit [b6127d0](#).

Cyfrin: Verified.

7.4.6 Explicit payouts[0] = 0 and payouts[1] = 0 writes a zero value

Description: ChainlinkAdapter::_constructPayouts explicitly writes 0 to an already-zero memory slot — dead code.

```
88:         payouts[0] = 0;
89:         payouts[1] = 1;
98:         payouts[0] = 1;
99:         payouts[1] = 0;
```

Impact: Dead code; minor gas waste.

Recommended Mitigation: Drop redundant zero writes.

Predict.fun: Acknowledged; yes it is dead code but we're trying to avoid ambiguity.

7.4.7 ChainlinkReceiverBase::onReport storage re-reads of optional permission fields

Description: ChainlinkReceiverBase::onReport reads s_expectedAuthor up to 3 times on the success path and s_expectedWorkflowName up to 2 times. Warm SLOADs (~100 gas each) but caching clarifies control flow. Overlaps with the G-3 issue.

Impact: Minor warm-SLOAD gas waste; modest readability hit.

Recommended Mitigation:


```

address expectedAuthor = s_expectedAuthor;
bytes10 expectedName = s_expectedWorkflowName;
if (expectedAuthor != address(0) || expectedName != bytes10(0)) {
    if (expectedAuthor != address(0) && workflowOwner != expectedAuthor) revert
        ↪ InvalidAuthor(workflowOwner, expectedAuthor);
    if (expectedName != bytes10(0)) {
        if (expectedAuthor == address(0)) revert WorkflowNameRequiresAuthorValidation();
        if (workflowName != expectedName) revert InvalidWorkflowName(workflowName, expectedName);
    }
}
}

```

Predict.fun: Fixed in commit [a0aa366](#) which:

- refactored `s_expectedAuthor` and `s_expectedWorkflowName` into a mapping and now set in `setIsWorkflowAuthorAndNameValid`
- in `onReport` replaced the previous code with `if (!isWorkflowAuthorAndNameValid[workflowOwner][workflowName])` that reads the owner workflow and name once

Cyfrin: Verified.